

Résolution automatique de théorèmes par SAT

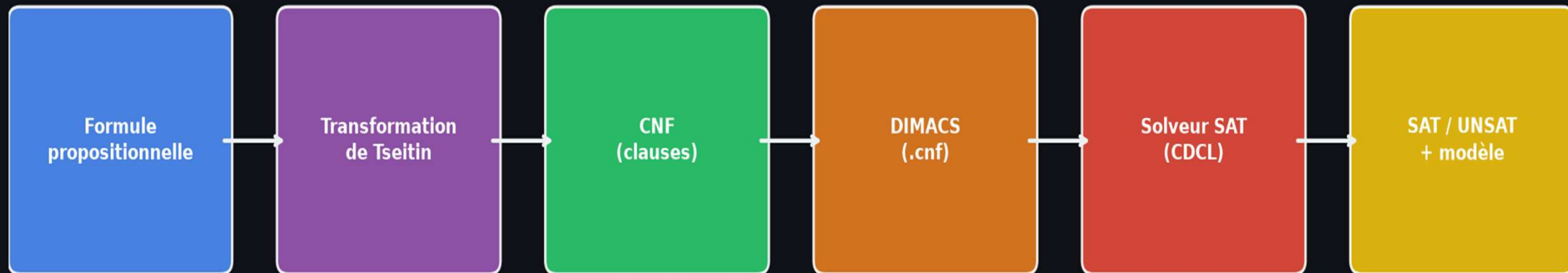
Projet B1 — IA Symbolique & Programmation par Contraintes

EPITA 2026

Gurvan Estable — Kevin Lubert — Joris Bely

De la formule à la décision : pipeline complet

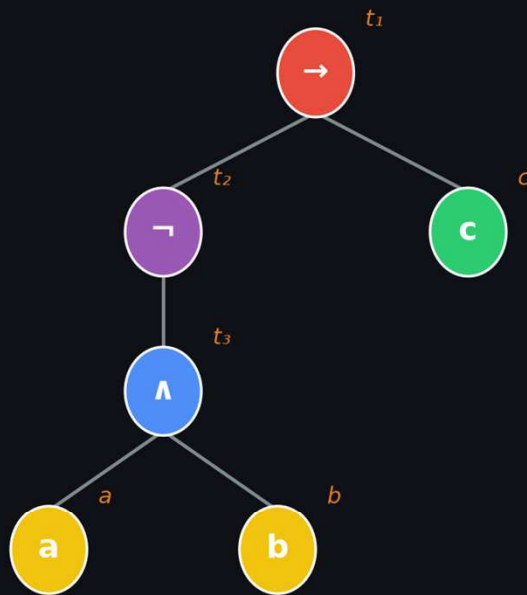
Pipeline complet : de la formule à la décision



Taille CNF linéaire en la taille de la formule (Tseitin)

Transformation de Tseitin — CNF en taille linéaire

Formule : $(a \wedge b) \rightarrow c$



Chaque nœud $\rightarrow$ variable auxiliaire t_i

Taille CNF = $O(|\text{formule}|)$ vs naïf = $O(2^n)$

Clauses CNF générées

$t_3 \leftrightarrow (a \wedge b)$

$\neg t_3 \vee a$

$\leftarrow t_3 \text{ implique } a$

$\neg t_3 \vee b$

$\leftarrow t_3 \text{ implique } b$

$t_3 \vee \neg a \vee \neg b$

$\leftarrow a \wedge b \text{ implique } t_3$

$t_2 \leftrightarrow \neg t_3$

$\neg t_2 \vee \neg t_3$

$\leftarrow t_2 \text{ implique } \neg t_3$

$t_2 \vee t_3$

$\leftarrow \neg t_3 \text{ implique } t_2$

$t_1 \leftrightarrow (t_2 \vee c)$

$\neg t_1 \vee t_2 \vee c$

$\leftarrow t_1 \text{ implique } t_2 \vee c$

$t_1 \vee \neg t_2$

$\leftarrow t_2 \text{ implique } t_1$

$t_1 \vee \neg c$

$\leftarrow c \text{ implique } t_1$

Racine assertée

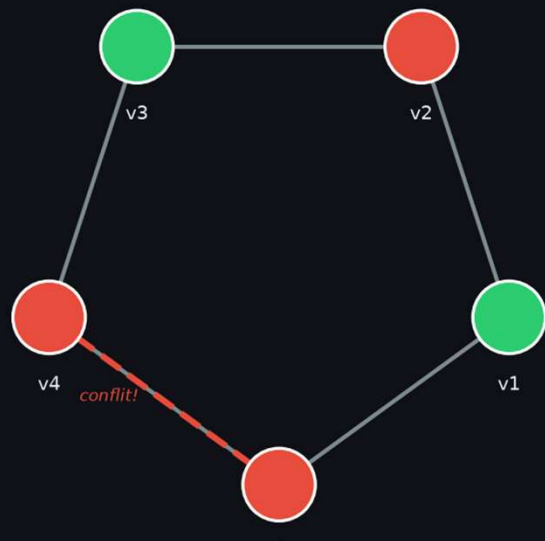
(t_1)

$\leftarrow$ force la racine à vrai

Encodage — Graph Coloring (coloriage de graphe)

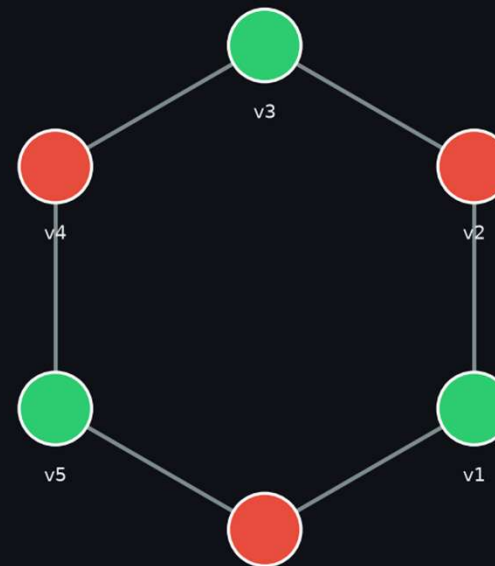
Encodage : Graph Coloring (2 couleurs sur cycles)

**C₅ — cycle impair (5 sommets)
2-coloriage**



UNSAT ✗ (conflict v0-v4 même couleur)

**C₆ — cycle pair (6 sommets)
2-coloriage**



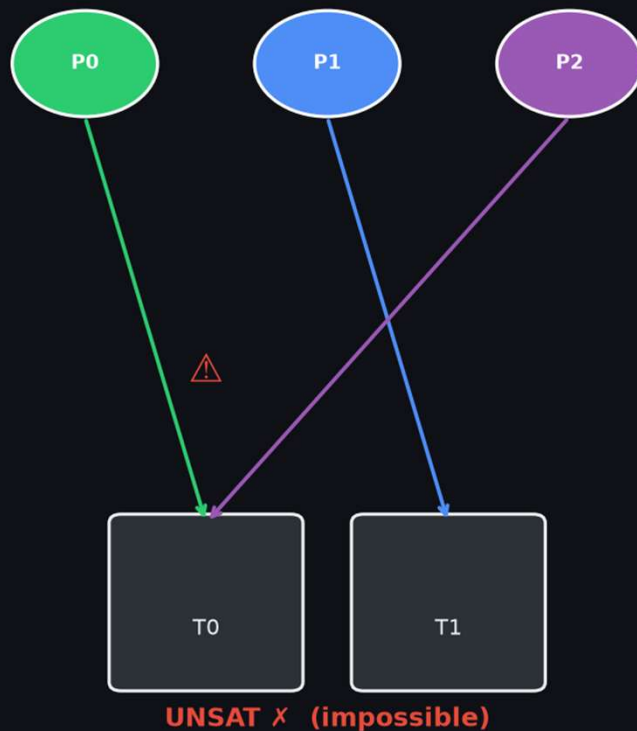
SAT ✓ (coloriage valide)

- Variable x_{v_c} : le sommet v prend la couleur c
- Contraintes : au moins une couleur / au plus une / pas de voisins identiques
- Cycle impair → UNSAT | Cycle pair → SAT

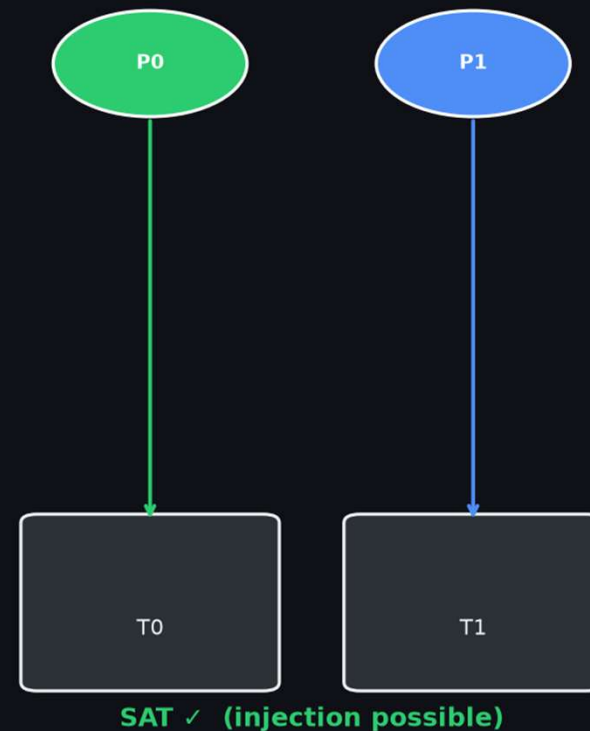
Encodage — Principe des tiroirs (Pigeonhole)

Encodage : Principe des tiroirs (Pigeonhole)

3 pigeons → 2 trous



2 pigeons → 2 trous

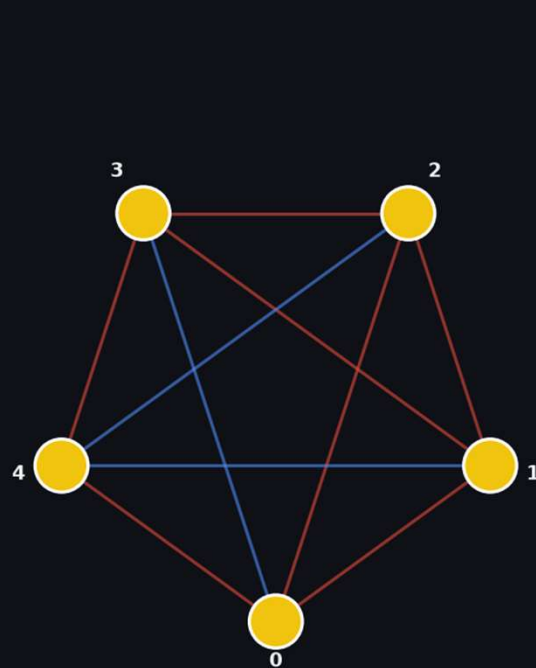


- Variable x_{p_h} : le pigeon p est dans le trou h
- $n+1$ pigeons dans n trous → toujours UNSAT
- Instance classique pour tester la robustesse des solveurs

Encodage — Ramsey $R(3,3) = 6$

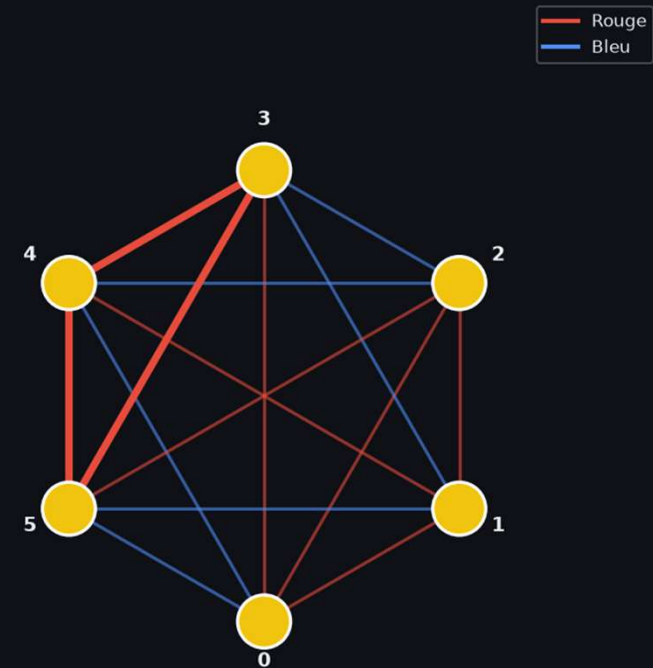
Encodage : Ramsey $R(3,3)$ — arêtes bicoloriées

K_5 — graphe complet à 5 sommets



SAT ✓ (pas de triangle monochrome)

K_6 — graphe complet à 6 sommets



UNSAT ✗ (triangle mono inévitable $\rightarrow R(3,3)=6$)

- Variable e_{u,v_red} : l'arête (u,v) est rouge (sinon bleue)
- Pour chaque triangle : interdit tout-rouge ET interdit tout-bleu
- $K_5 \rightarrow \text{SAT } (R(3,3) > 5) \mid K_6 \rightarrow \text{UNSAT } (R(3,3) \leq 6) \rightarrow R(3,3) = 6$

Format DIMACS — standard universel SAT

Format DIMACS — exemple graph coloring (triangle 3-col)

```
c Coloriage du triangle K3 avec 3 couleurs
c Variables : x_v{sommet}_c{couleur}
c 3 sommets x 3 couleurs = 9 variables
p cnf 9 21

c --- Sommet 0 a au moins une couleur ---
1 2 3 0
c --- Sommet 0 a au plus une couleur ---
-1 -2 0
-1 -3 0
-2 -3 0

c --- Sommet 1 a au moins une couleur ---
4 5 6 0
c --- Sommet 2 a au moins une couleur ---
7 8 9 0

c --- Arête (0,1) : même couleur interdite ---
-1 -4 0
-2 -5 0
-3 -6 0

c --- Arête (0,2) et (1,2) similaires... ---
...
```

Structure d'un fichier DIMACS

c ... Commentaire (ignoré par le solveur)

p cnf V C En-tête : V variables, C clauses

1 2 3 0 Clause : $(x_1 \vee x_2 \vee x_3)$ — terminée par 0

-1 -2 0 Clause : $(\neg x_1 \vee \neg x_2)$

variable positive Littéral positif x_i

variable négative Littéral négatif $\neg x_i$ (précédé de -)

Standard universel : compatible avec tout solveur SAT

Solveurs SAT — DPLL (fallback) vs CDCL (PySAT)

Architecture des solveurs SAT : DPLL (fallback) vs CDCL (PySAT)

DPLL (implémentation locale)

Propagation
unitaire (UP)

Élimination
littéraux purs

Branchement
(fréquence)

Retour
arrière naïf

+ Aucune dépendance

+ Compréhensible

– Lent sur grandes instances

CDCL (glucose3 / cadical153 / lingeling)

Propagation
unitaire (BCP)

Apprentissage
de clauses

Branchement
VSIDS

Backjump
non-chronologique

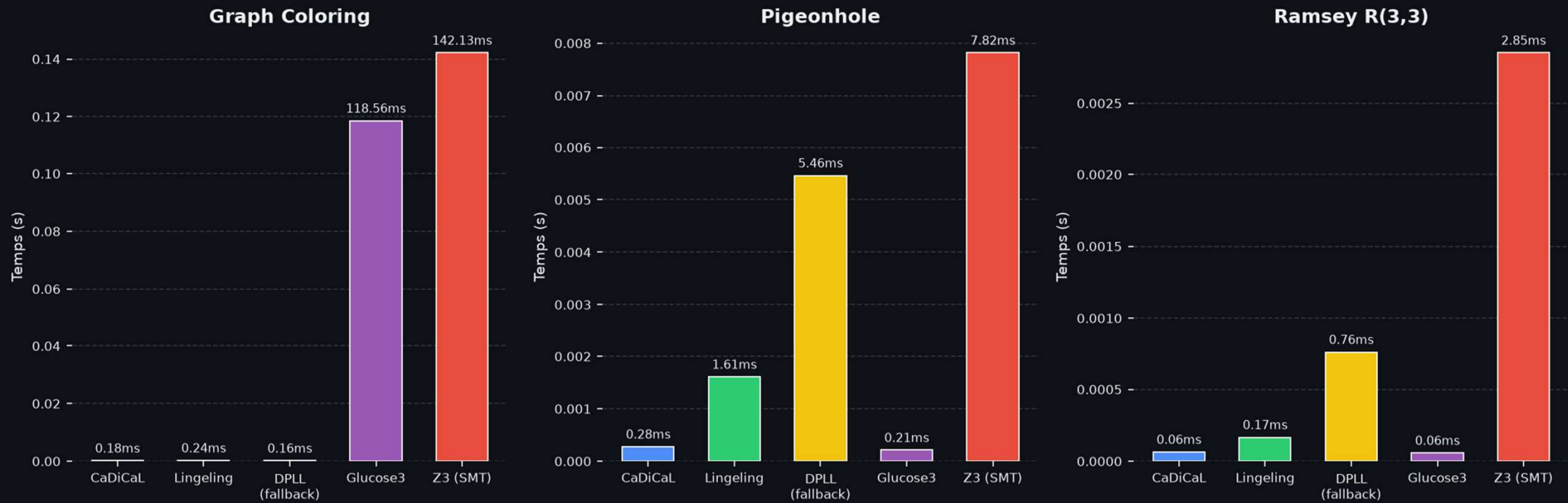
+ Très efficace (prod)

+ Certif. UNSAT (DRAT)

– Boîte noire

Résultats — Benchmark SAT vs SMT(1)

Benchmark : temps moyen de résolution (secondes)



Résultats — Benchmark SAT vs SMT(2)

SAT vs SMT — Comparaison des approches

Critère	SAT + Encodage manuel	SMT (Z3)
Niveau d'abstraction	Booléen (bits explicites)	Entiers, réels, tableaux...
Encodage	Manuel + transformation Tseitin	Déclaratif, proche du problème
Format sortie	DIMACS (.cnf) universel	API Python Z3
Vérification preuve	DRAT/LRAT (drat-trim)	Limité selon la théorie
Performance (benchmark)	~0.0002s (CaDiCaL sur pigeonhole)	~0.008s (Z3 sur pigeonhole)
Portabilité	Tout solveur DIMACS-compatible	Librairie Z3 requise
Lisibilité du modèle	Conclusion : SAT = contrôle total + preuve formelle SMT = productivité + lisibilité	
	Mapping variable→sémantique	Directe (int, bool nommés)

Bilan — Projet B1

Bilan du projet B1



Transformation de Tseitin

Taille CNF linéaire, sans explosion combinatoire



3 familles encodées

Graph Coloring, Pigeonhole, Ramsey $R(3,3)$



4 solveurs benchmarkés

glucose3, CaDiCaL, Lingeling, DPLL fallback



Comparaison SAT / SMT

Z3 intégré pour toutes les familles



Validation à double niveau

CNF brute + interprétation métier



Certificats UNSAT

Export DRAT possible, vérif. dépend de l'environnement



Encodages cardinalité

Pairwise utilisé (plus simple, moins compact)

10

fichiers source

3

familles de problèmes

100%

instances résolues

<1ms

temps moyen SAT